任务 1：数据库与查询条件基础说明

1. 页面中的输入为什么会与数据库查询有关

因为网站要记住、验证或者展示用户相关的东西，比如登录的时候要查有没有这个账号，或者搜索商品的时候要去商品表里找匹配的记录。没有数据库，网站基本上就是个死页面，输什么都没反应。所以页面输入最终一定会传到后端，后端再拿着它去问数据库

2. 一个最小业务链路是什么

在登录页输入用户名和密码，点一下登录，这就是输入条件。后台收到后，拼一句 SQL 去数据库查，比如“select * from 用户表 where 名字=‘小明’ and 密码=‘123’”，这就是查询。数据库告诉后台：“找到了，一行”，后台就告诉浏览器“登录成功”。这就叫一个完整的小闭环。

3. 用户输入是如何进入查询条件的

因为后端程序是“拼接”进去的。

程序员写了一个 SQL 模板，比如 select * from users where name = ‘[这里放输入]’，

然后程序拿到页面上用户填的内容，直接把内容的字符串替换到那个占位的位置。所以如果你填了“小红”，那最终 SQL 里就变成 name = ‘小红’。输入就是通过这种字符串拼接的方式，硬塞进查询条件里的。

4. 为什么这为 SQL 注入创造了可能性

因为后端程序是“拼接”进去的。程序员写了一个 SQL 模板，比如 select * from users where name = ‘[这里放输入]’，

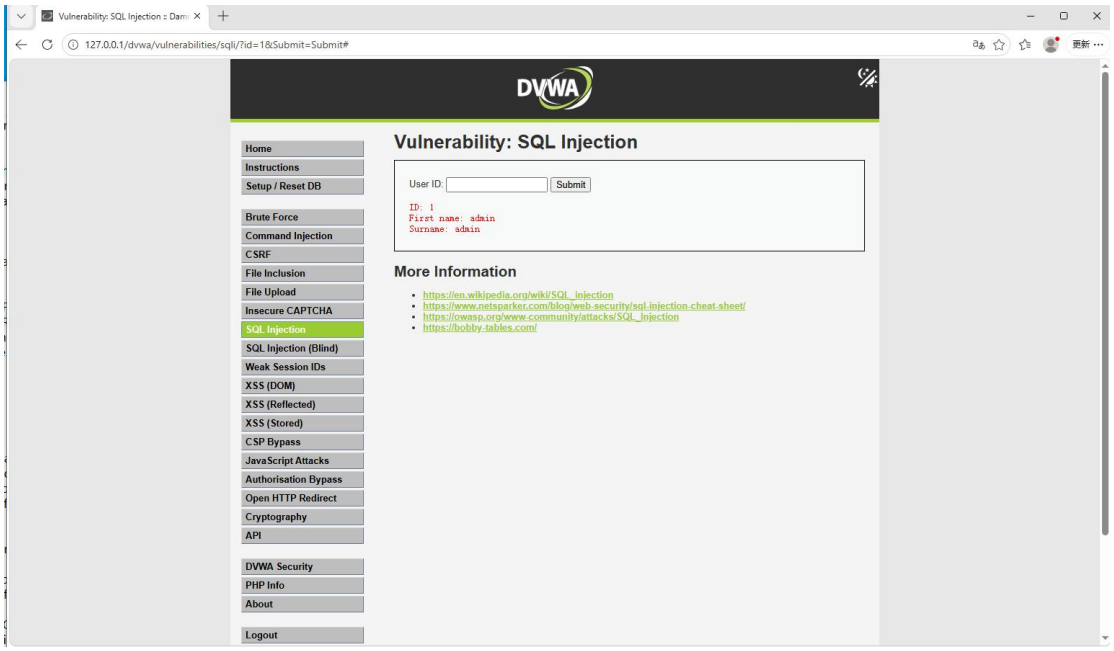
然后程序拿到页面上用户填的内容，直接把内容的字符串替换到那个占位的位置。

所以如果你填了“小红”，那最终 SQL 里就变成 name = ‘小红’。

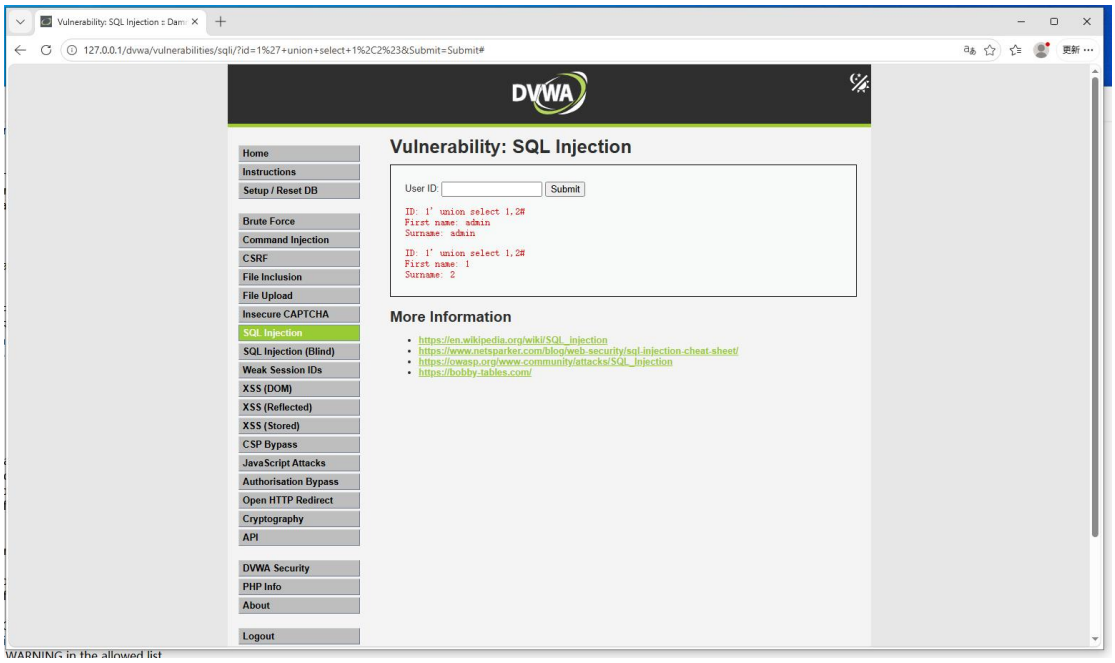
输入就是通过这种字符串拼接的方式，硬塞进查询条件里的。

任务 2：DVWA SQL Injection Low 现象记录

普通输入：



异常输入：



页面正常显示 ID: 1 的那条 admin 记录，但在下面又多出了一条奇怪的结果：

First name: 1，Surname: 2

正常查询再怎么改 ID，也不可能凭空造出一个 first name=1, surname=2 的记录来。

这种差异说明：攻击者已经能够直接控制 SQL 返回的数据结构和内容，不仅仅是改变筛选条件，而是在向数据库“植入”我们自己定义的结果集。

任务 3：机制分析与差异比较

1. 输入如何进入查询结构

最典型的路径是：

输入 → 拼接 → 解析 → 执行 → 回显/行为差异

举个例子：

后端代码里写了一条 SQL 模板 "SELECT * FROM users

WHERE id = ’″ + 用户输入 + ″’″。

用户在页面上输入 1，这个字符串就被直接拼到模板里，变成 ... id = ’1’。

然后数据库解析这条 SQL，看到单引号里面的内容当作字符串值，再执行，最后把结果回显到页面上。

这就是输入进入查询结构的全过程——通过字符串拼接，没有任何隔离。

2. 为什么根因是“结构被污染”而不是“字符本身危险”

单个字符本身没有任何攻击能力。

比如 ’ 在数据库里就是个普通符号，你存一个名字叫 O’Neil 是合法的。

真正出问题的是：这个 ’ 出现在一个特定位置，它提前闭合了原本的字符串边界，导致后面跟着的 OR 等关键词变成了 SQL 指令的一部分。

3. Low 与 Medium 的差异在哪里

DVWA 的 Low 级别：直接拼接，不做任何处理。

Medium 级别：用了 mysql_real_escape_string() 函数，把单引号、双引号等转义掉。

差异看起来是“Medium 更安全”，但我亲手试了之后发现：Medium 级别的查询改成了数字型（WHERE user_id = $id，不带引号）。

那么我输入 1 and 1=1 就能成功注入，根本用不到引号，转义函数完全绕过去了。

所以 Low 和 Medium 的本质差异不是“转义与否”，而是：

Low 对任何输入不加区分地拼接

Medium 试图用转义堵住引号这条路，但没改查询的数据类型，于是数字型注入依然畅通

4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里

普通注入：页面可能会直接显示数据库中的数据，或者抛出详细的数据库报错信息

盲注：页面不返回具体数据，也不报错。你只能通过“页面变了/没变”、“多了一行/少了一行”、“响应快了/慢了”这类的二值信号去推断信息

所以差异不在攻击原理，而在服务器对你的“反馈形式”。

5. 为什么盲注和普通注入根因相同，但证据形式不同

因为应用程序对错误的处理方式不同：

有些程序直接把查询结果或报错信息打印到页面上（普通注入如鱼得水）

有些程序把所有错误吞掉，只返回一个固定页面（比如“查询无结果”），攻击者只能通过布尔条件或时间延迟来一点

点试探

任务 4：SQL 注入防护清单与解释

针对 SQL 注入漏洞，通常需要从多个层面部署防护措施，其中最核心且唯一能从根本上解决问题的是参数化查询（也叫预编译语句）。它的原理是把 SQL 语句的结构和数据彻底分离：先让数据库编译 SQL 模板（比如 SELECT * FROM users WHERE id = ?），然后再把用户输入作为参数传进去。这样一来，无论用户输入什么内容，数据库都只会把它当成纯数据，永远不会被解析成 SQL 指令。参数化查询控制的是核心代码层/数据访问层，直接针对根因——“输入进入查询结构”。但它不能替代输入校验，比如参数化查询无法防止用户传入超长字符串或非法格式，那些属于业务层面的问题。

第二层是输入校验，优先采用白名单方式。例如，如果某个字段只允许数字，那么后端直接拒绝任何包含字母、引号或 SQL 关键字的输入。它的作用是缩小攻击面，让恶意请求在到达数据库之前就被拦截。输入校验控制的是应用层（前后端），属于暴露面缩减。但它不能替代参数化查询，因为校验规则总有遗漏（比如搜索框需要支持各种字符），一旦规则有漏洞，注入依然可能发生。

第三层是错误处理，核心原则是绝对不要把原生数据库错误

信息（如语法错误、表名、字段名）直接返回给用户。正确做法是只返回“系统繁忙”或“查询无结果”这类通用提示。这样做的目的是隐藏攻击线索，防止攻击者通过报错回显收集数据库结构信息。错误处理控制的是应用层（后端配置）和输出层，同样是暴露面缩减。但它不能替代参数化查询，因为注入本身仍然可能发生，只是攻击者得不到报错反馈，还得通过盲注等更麻烦的方式继续试探。

第四层是最小权限原则，即应用程序连接数据库时，使用一个权限极低的专用账号，只对必要的表执行必要的操作（比如只做 SELECT，不做 DROP、INSERT、INTO OUTFILE 等）。这样即使 SQL 注入发生了，攻击者能造成的破坏也非常有限，例如无法删表、无法写文件、无法修改数据。最小权限控制的是数据库层/权限管理层，属于后果范围缩减——它不阻止注入发生，但能大幅降低损失上限。但这不能替代参数化查询，因为注入依然可以进行数据读取，如果该表存有敏感信息，还是可能泄露。

此外，还可以补充部署 Web 应用防火墙（WAF），它在网络流量层拦截已知的恶意请求特征。WAF 属于暴露面缩减，可以作为补充防线，尤其适合无法快速修改代码的历史遗留系统。但它依赖规则更新，且很容易被编码、分块传输等手法

绕过，绝不能替代参数化查询。

总结来说，真正能控制根因的只有参数化查询；输入校验和错误处理是在攻击路径上缩减暴露面；最小权限和 WAF 则是缩小攻击成功后的影响范围或提供额外过滤。这四类措施必须配合使用，缺一不可，但最关键的永远是第一道防线：不让用户输入有机会污染 SQL 结构。